



Operaciones Terminales en Streams de Java

Cómo obtener resultados a partir de un Stream

En esta sesión aprenderemos cómo transformar nuestros Streams en resultados concretos utilizando operaciones terminales, el paso final en el procesamiento de datos con la API Stream de Java.

An illustration at the top of the slide shows a teal background with a dark green winding path. On the left, a pink dam with four small rectangular openings is partially submerged. A white arrow points from a speech bubble to the dam. To the right of the dam, a pink rectangular block is partially submerged. Further right, a pink sphere with a red curved line is partially submerged. A white arrow points down towards the sphere. The speech bubble is orange with rounded corners and contains the text 'Closed the Stream' in orange.

**Closed
the Stream**

¿Qué son las operaciones terminales?

Las operaciones terminales son aquellas que:

- Transforman el Stream en un resultado final concreto
- Cierran el Stream, impidiendo su reutilización posterior
- Activan la ejecución de todas las operaciones intermedias acumuladas (evaluación perezosa o lazy evaluation)

Hasta que no se invoca una operación terminal, ninguna operación intermedia se ejecuta realmente.

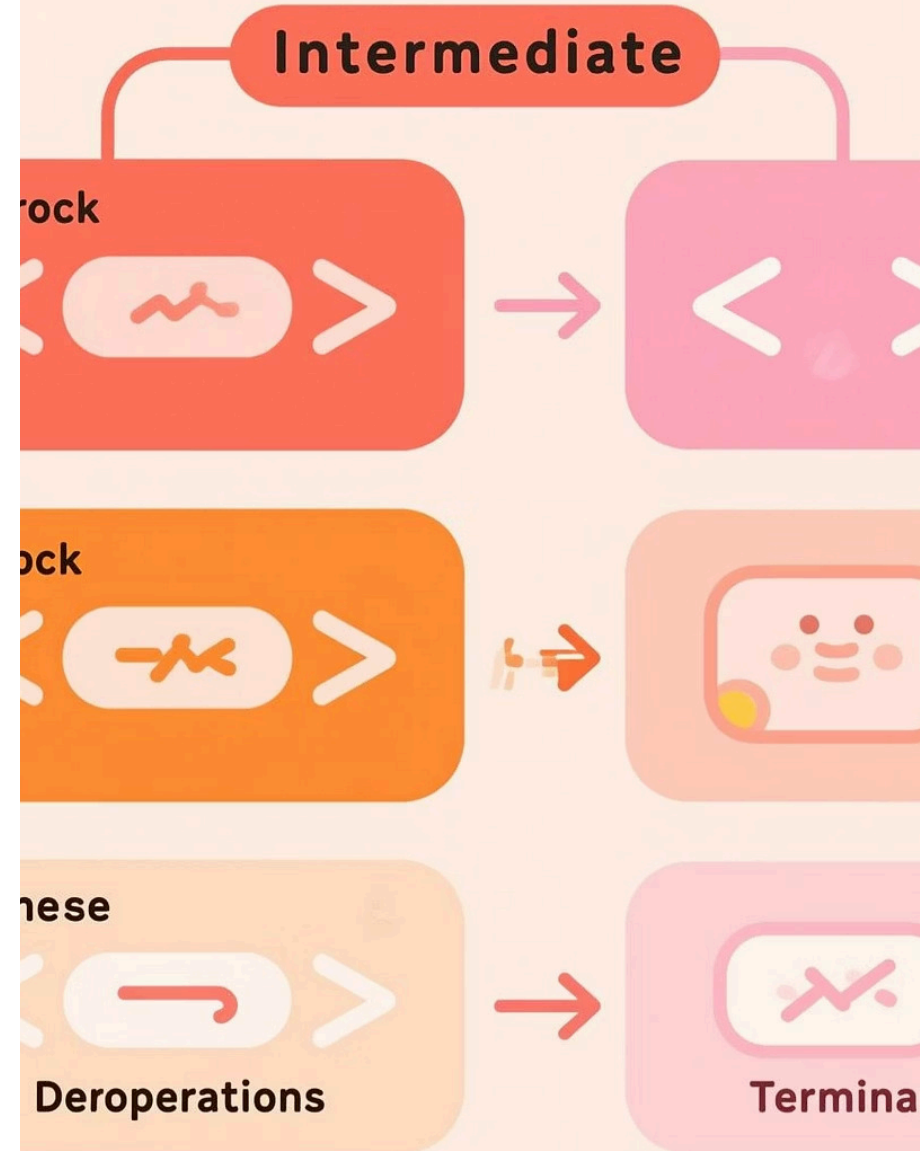
Operaciones Intermedias vs. Terminales

Operaciones Intermedias

- Devuelven otro Stream
- Se pueden encadenar múltiples veces
- No se ejecutan hasta que haya una operación terminal
- Ejemplos: `map()`, `filter()`, `sorted()`

Operaciones Terminales

- Devuelven un resultado concreto (no un Stream)
- Solo puede haber una por Stream
- Provocan la ejecución de toda la cadena
- Ejemplos: `forEach()`, `collect()`, `reduce()`, `count()`



forEach()

Ejecuta una acción sobre cada elemento del Stream y no devuelve ningún valor.

```
// Imprimir todos los nombres en la consola
List nombres = List.of("Ana", "Carlos", "Elena", "David");
nombres.stream()
    .forEach(System.out::println);
```

Ideal para:

- Imprimir elementos a la consola
- Ejecutar acciones laterales (side effects) con cada elemento
- Consumir los resultados del Stream

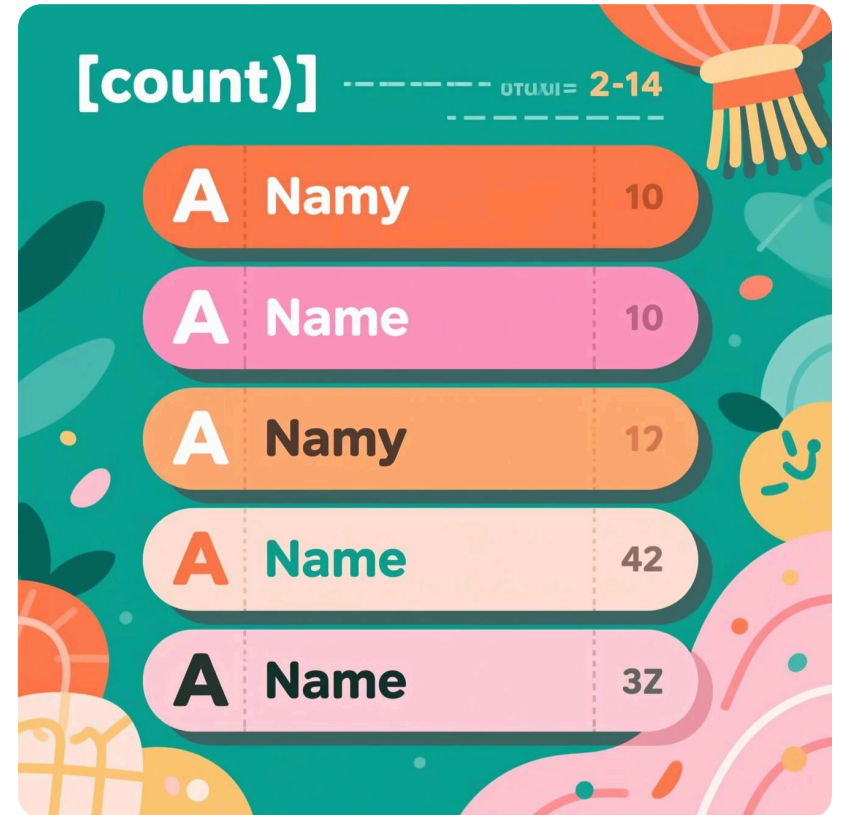


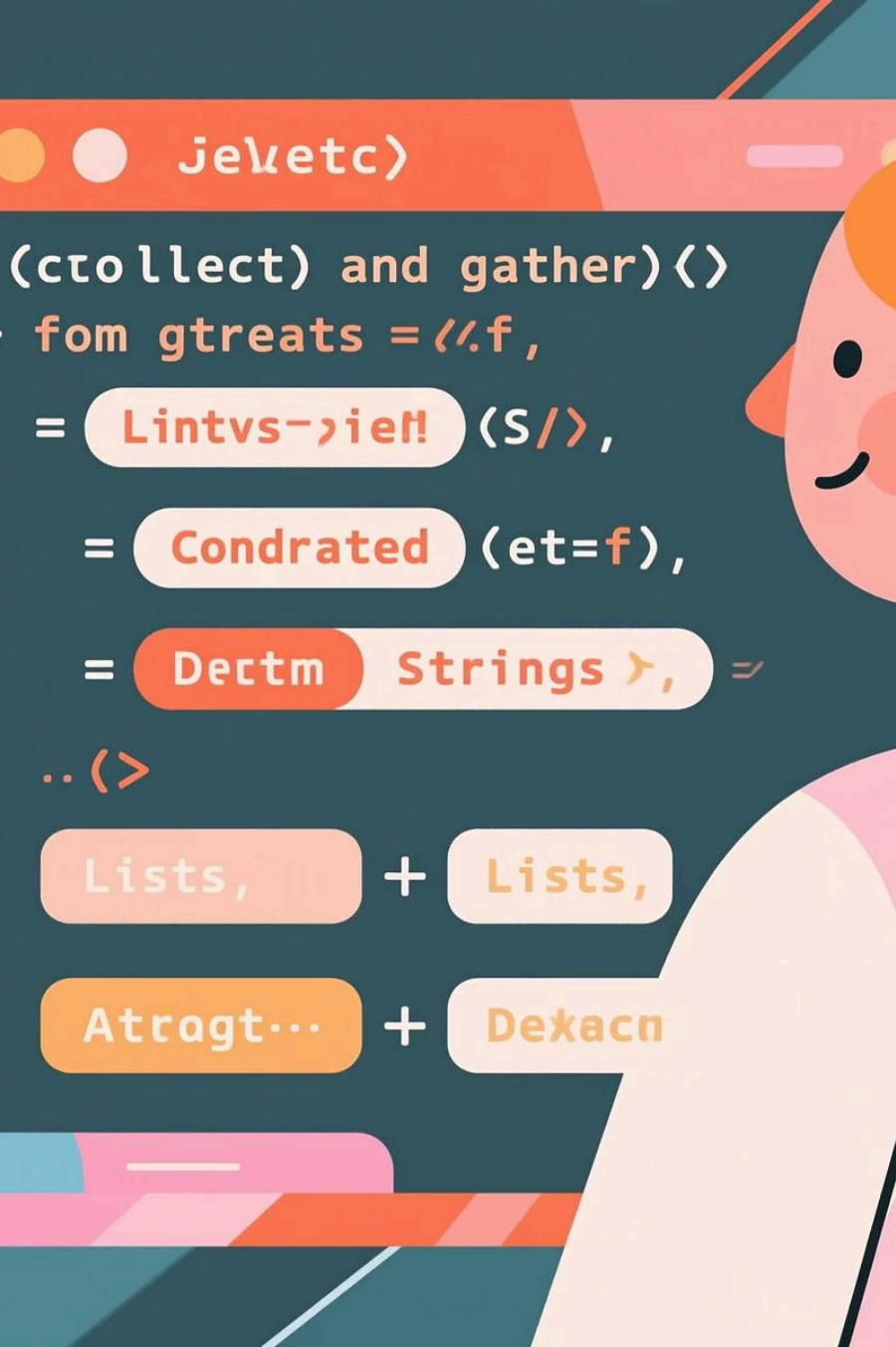
count()

Devuelve la cantidad de elementos presentes en el Stream como un valor `long`.

```
List nombres = List.of("Ana", "Carlos",  
                        "Elena", "David");  
  
// Contar cuántos nombres hay  
long cantidad = nombres.stream().count();  
System.out.println("Total: " + cantidad);  
  
// Contar nombres que empiezan con 'A'  
long empiezanConA = nombres.stream()  
    .filter(n -> n.startsWith("A"))  
    .count();
```

Una operación terminal simple pero muy útil para análisis básicos.





collect()

Recolecta los elementos del Stream en una colección u otra estructura de datos.

```
List nombres = List.of("ana", "carlos", "elena", "david");
```

```
// Convertir a mayúsculas y recolectar en una nueva lista
```

```
List mayusculas = nombres.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

```
// Otras estructuras de destino
```

```
Set conjuntoNombres = nombres.stream()
    .collect(Collectors.toSet());
```

```
String concatenados = nombres.stream()
    .collect(Collectors.joining(", "));
```

Es una de las operaciones terminales más versátiles y potentes gracias a la clase `Collectors`.

reduce()

Combina todos los elementos del Stream en un único resultado mediante una operación acumulativa.

```
List numeros = List.of(2, 4, 6, 8);

// Sumar todos los números
int suma = numeros.stream()
    .reduce(0, (a, b) -> a + b);
// Equivalente a:
int suma2 = numeros.stream()
    .reduce(0, Integer::sum);

// Encontrar el máximo
Optional maximo =
numeros.stream()

.reduce(Integer::max);
```

0

Valor inicial

Valor de identidad para empezar la
operación

$(a,b) \rightarrow a+b$

Función acumuladora

Combina elementos progresivamente

20

Resultado final

Tras aplicar reducción a $2+4+6+8$



Ejemplo integrador

Vamos a trabajar con una lista de números para mostrar diferentes operaciones terminales:

```
List numeros = List.of(2, 4, 6, 8, 3, 5, 7, 9);
```

```
// 1. Contar elementos
```

```
long cantidad = numeros.stream().count();
```

```
System.out.println("Total de números: " + cantidad);
```

```
// 2. Sumar todos los valores
```

```
int suma = numeros.stream().reduce(0, Integer::sum);
```

```
System.out.println("Suma total: " + suma);
```

```
// 3. Agrupar por pares e impares
```

```
Map> agrupados = numeros.stream()
```

```
.collect(Collectors.groupingBy(
```

```
n -> n % 2 == 0 ? "Pares" : "Impares"));
```

```
System.out.println(agrupados);
```


Resumen: Operaciones Terminales



Finalizan el Stream

Las operaciones terminales cierran el flujo de datos y producen un resultado concreto, activando todas las operaciones intermedias anteriores.



Principales operaciones

- `forEach()` → ejecutar acción
- `count()` → cantidad de elementos
- `collect()` → recolectar en estructuras
- `reduce()` → combinar en un único valor



Próximos temas

En la siguiente sesión exploraremos los **Collectors avanzados** para obtener resultados más sofisticados a partir de nuestros Streams.

y- Terminal

